

Privacy-Preserving Authentication Scheme for Connected Electric Vehicles Using Blockchain and Zero Knowledge Proofs

David Gabay , Kemal Akkaya, *Senior Member, IEEE*, and Mumin Cebe

Abstract—With the increasing interest in connected vehicles along with electrification opportunities, there is an ongoing effort to automate the charging process of electric vehicles (EVs) through their capabilities to communicate with the infrastructure and each other. However, charging EVs takes time and thus in-advance scheduling is needed. As this process is done frequently due to limited mileage per charge on EVs, it may expose the locations and charging pattern of the EV to the service providers, raising privacy concerns for their users. Nevertheless, the EV still needs to be authenticated to charging providers, which means some information will need to be provided anyway. While there have been many studies to address the problem of privacy-preserving authentication for vehicular networks, such solutions will be void if charging payments are made through traditional means. In this paper, we tackle this problem by utilizing distributed applications enabled by Blockchain and smart contracts. We adapt zero-knowledge proofs to Blockchain for enabling privacy-preserving authentication while removing the need for a central authority. We introduce two approaches, one using a token-based mechanism and another utilizing the Pederson Commitment scheme to realize anonymous authentication. We also describe a protocol for the whole process which includes scheduling and charging operations. The evaluation of the proposed approaches indicates that the overhead of this process is affordable to enable real-time charging operations for connected EVs.

Index Terms—Electric vehicles, blockchain, zero knowledge proofs, Pederson commitment.

I. INTRODUCTION

NEW generation vehicles are becoming much smarter at various levels, including but not limited to infotainment, brake assistance, adaptive cruise control, parking assistance, driver fatigue detection, and more [1].

This is in particular due to recent developments on *Vehicle to Infrastructure* communication (V2I), which enables a smart car to communicate with its environment (i.e., determine traffic details, communicate with service providers, etc.) as well as *Vehicle-to-Vehicle* (V2V) communication that enables safety

and social applications between vehicles (i.e., forward collision warning, lane change warning, etc.). With V2V and V2I it is easy to imagine scenarios where the human driver is out of the picture. Ultimately, there is a lot of potential applications due to such developments in vehicular networks [2].

One subclass of smart vehicles are Electric Vehicles (EVs) which operate via electricity as opposed to gasoline powered vehicles. With a single charge, EVs can operate anywhere from 50-200 miles [3], which means they will need frequent charging. They can rely on V2I and V2V to communicate with other EVs and EV Charging Service providers (EVSP) to schedule charging. With the advancements in autonomous vehicle technologies, they are expected to conduct charge scheduling and even charging automatically. Even charging through another EV would be possible in this manner, which will enable social communication among EVs [3].

However, frequent charging would bring privacy issues as studied in the literature. Specifically, in the case of charge scheduling, the service providers would have access to all of the EVs information since this is similar to a client-server model. For instance, over time the service providers can learn a lot about their clients including their personal data, location patterns, and habits. This data can be shared with other third parties such as marketers, insurance companies, etc. Also, if any payment is made for the service, that can raise privacy concerns, as the payment information will still be collected. These issues with a centralized system lead to the implementation of various privacy laws as in [4], but this does not address the problem as full enforcement of these policies is not easy.

An alternative to address the privacy issue that is being considered is moving to a decentralized architecture, particularly Blockchain technologies that utilize the distributed ledger architecture. When considering our application, Blockchain will provide the following key advantages that make it a viable option:

- *Prevents single point of failure issues.* Service providers that have data breaches are often due to some single point of failure where the customers data is breached due to a single security measure failing. With Blockchain, the security of the application is the responsibility of all the participants, and all data is stored with the participants.
- *Anonymous payments.* Through the use of cryptocurrencies payment information will not be exposed to service providers. For example, payment information like credit cards is not stored for future service payments.
- *Distributed trust.* With all transactions publicly accessible and verified by the public, it is difficult to make false

Manuscript received September 15, 2019; revised December 16, 2019; accepted February 10, 2020. Date of publication March 2, 2020; date of current version June 18, 2020. This paper was presented in part at the 44th IEEE Conference on Local Computer Networks, Osnabrück, Germany, October 14–17, 2019. The review of this article was coordinated by Prof. H. Li. (*Corresponding author: David Gabay.*)

The authors are with the Department of Electrical and Computer Engineering, Florida International University, Miami, Florida 33174 USA (e-mail: dgaba002@fiu.edu; kakkaya@fiu.edu; mcebe@gmail.com).

Digital Object Identifier 10.1109/TVT.2020.2977361

transactions. In this way, it keeps both the service provider and the consumer honest and accountable. Additionally, Blockchain's immutable characteristic provides both parties with undeniable evidence of transactions.

Applications that are run on such a distributed ledger are now commonly referred to as *Decentralized Applications (DApps)*. There are several examples such as supply chain management [5], decentralized voting, Internet of Things [6], digital rights management (DRM) [7], auto insurance claims and adjudication [8] which aim to mitigate the risks accompanied by centralized systems. EV Charging can also be moved to a distributed architecture with the use of smart contracts on a distributed ledger. In the case of EV Charging applications, the EV communicates with a smart contract, and with the EVSP before it can be allowed to charge at a charging station. Even though a distributed architecture can support these operations through smart contracts, there will still be a challenge. This is because there should be a mechanism to allow only registered EVs to access the service through authentication. Second, this authentication mechanism should also ensure privacy.

To address these issues, in this paper, we propose an efficient and feasible framework that allows an EV user to request charging scheduling from an EVSP in an Ethereum Blockchain [9] application, while ensuring the privacy of the EV user. To this end, we utilize the concept of Zero-Knowledge Succinct Non-Interactive Argument of Knowledge, commonly known as zkSNARKs [10] which aims to provide an efficient variant of a zero-knowledge proofs [11]. zkSNARKs allows a person to prove to another person the correctness of a statement without revealing the contents of the proof in one message.

In our proposed framework, we use zkSNARKs efficiently by having a single contract verify every EV using the decentralized application, as opposed to having a smart contract generated for each EV. Once verified by the smart contract without being identified, we use an Ethereum token system to issue tokens to that EV. The tokens will be used for scheduling and charging service requested through a near random address, hereafter referred to as a pseudonym. The token approach will ensure the authentication and charging operations are kept separate and thus EVs remain anonymous to both the EVSP and the public entities using the Blockchain.

However, since this approach still requires writing to Blockchain for scheduling and charging services which brings some overhead, we introduce an improvement to our scheme to eliminate this overhead. Specifically, we propose using the *Pederson Commitment* scheme [12] for authentication as well as charge scheduling purposes. In the Pederson approach, the commitment is both binding and hiding, meaning nobody can infer who submitted the commitment and only the committer can prove he/she submitted the commitment. We still follow the same protocol but the steps for token-based operations are eliminated with the help of the Pederson scheme. In this way, we are able to eliminate access to Blockchain for charge scheduling and charging and hence improve efficiency and cost.

We implemented both approaches of the proposed framework and assessed their feasibility and performance. We showed that our proposed model provides anonymity for the EV (user) during communication with the EVSP and charging stations. Our framework's underlying protocol Zokrates [13], provides strong proofs that enable only legitimate users to generate valid

proofs. We also show that the cost and computation time for the framework on the Ethereum Blockchain are acceptable ranging from \$0.23 to deploy a smart contract and \$0.13 to verify the proof. Also, scheduling a charging slot for the EV can be done in less than 38 seconds when using the Pederson approach.

The rest of this paper is organized as follows: We summarize the related literature next. In Section III, we provide background information related to DApps, Zero-knowledge proofs, the Pederson commitment scheme, and the current vehicle communication model. In Section V, we present our token-based approach using zkSNARKs and Blockchain, followed by our improved Pederson approach in Section VI. In Section VII, we discuss the security and privacy analysis of our framework. In Section VIII, we evaluate our proposed framework, comparing both approaches, and conclude in Section IX.

II. RELATED WORK

A. Traditional Privacy Approaches in EV Charging

There have been many works for privacy preservation of EVs in the past [14]. These focused on several approaches including homomorphic encryption, blind signatures, and pseudonyms to preserve either the location or identity privacy of the EVs.

Pseudonyms are widely used for privacy purposes. Basically, anonymous identities are assigned to EVs as also suggested in the IEEE 1609.2 standard [15]. However, these pseudonyms are assigned to EVs by a set of trusted authorities. Also, the mechanism, by design, allows the trusted authorities to track the EVs actions if desired. The suggested pseudonym mechanism in IEEE 1609.2 only protects the privacy of EV from untrusted parties. Thus, the pseudonym mechanism does not provide the desired privacy of the EV against all parties. In our case, we rely on the pseudonyms too but we ensure the privacy even against authorities/EVSP by utilizing zero-knowledge proofs.

Homomorphic encryption is used in various contexts but mainly to enable location privacy and data aggregation, if any [16]. The idea is to hide the information in the distance and other computations. In EV charging, this can apply to make schedules on the encrypted data but this would require fully homomorphic encryption which is very slow and limited incapability to deal with all search functions.

Blind signatures are another option to provide privacy during authentication. This is a cryptographic protocol [17] which allows the user to send a message which has been blinded to be signed by a third party. Therefore, the content of the message is not known to third parties. The signature on the blinded message can then be unblinded by the source and be attached to its future messages. While blind signatures were not directly used for EV charging, they have been employed for smart meter privacy protection from the utility in [18]. While these signatures are useful when used with anonymous IDs, the issue of payments for charging is still there which can expose privacy. Therefore, blind signatures need to be combined with a privacy-preserving payment system as well. Our proposed approach in this paper address both issues by combining the DApps and zero-knowledge proofs.

In [19], Jeong *et al.* propose a Blockchain-based authentication and billing system for EVs to charge. This work sets out to

prevent issues with the current distrustful hardware based charging meters by using Blockchain smart contracts. In this system, the authors use a public/private key authentication mechanism using a smart contract. In this case, charging is measured using software and is integrated with the Blockchain, which prevents any tampering by either parties.

B. EV Location Privacy in DApps

Location privacy is important when considering Decentralized applications since users interact on a public ledger where everyone can see the transactions. In [20], Amiri *et al.* uses the Blockchain and a Private Information Retrieval technique to design a smart parking system that preserves the users' privacy. In this way, a user could anonymously request a parking offer from another peer directly, and authenticate using short randomizable signatures.

In [21], Knirsch *et al.* designs a transparent EV charging framework with consideration of smart grid supply and demand. Knirsch uses blockchain to offer transparency and dynamic pricing to EVs. The EV is able to choose a charging slot based on its distance to the station and the offered price. Knirsch uses a SHA-256 hash of the requested charging period to hide the details of the charging period and commit to the schedule. EVs in this approach remain anonymous, and can only be identified by some randomly generated blockchain ID. This approach bypasses EV authentication and instead considers every EV an authorized customer of the EVSP. In our approach, we implement a method of authentication for the EV to prove they are authorized customers of the EVSP.

Similarly, in [22], Pustišek *et al.* provides an excellent model for autonomous charging station selection using blockchain smart contracts. However, in [22], the authors do not consider privacy of the EV, but rather focus on the protocol used to optimally select a charging station for that EV.

C. Zero-Knowledge Proofs and Blockchain

Zero-knowledge proofs have started being used in various DApps. Some of these approaches utilized zkSNARKs but to date, none of these studies targeted the EV charging problem. For instance, Zerocash [23] is the core of the cryptocurrency Zcash which utilizes zkSNARKs to preserve the privacy of transactions that occur on the Blockchain. [24] used zkSNARKs to develop a novel image authentication approach used to determine if a photograph taken represents the original and has not been modified. Other studies using zkSNARKs include [25] and [26] which are geared for ensuring the integrity of machine learning approaches and data sharing privacy, respectively.

In [27], Baza *et al.* uses Blockchain smart contracts along with Zero Knowledge Set Membership (ZKSM) Proofs to provide a decentralized ride-sharing service called "B-Ride". B-Ride preserves the users privacy (i.e. personal identity) and preserves users location privacy, all the while using the Blockchain to provide decentralization. Since B-Ride does not rely on a third party or service provider they implement a reputation model to track drivers based on ride completion.

D. Pederson Commitment Scheme for Authentication

The only relevant work in literature that utilizes the Pederson Commitment is reported in [28]. The authors used the Pederson Commitment scheme to hide the Biometric Identity (BID) of a user. Then, during authentication, the user can use a zero knowledge proof of knowledge protocol to prove to the SP that they are the correct user without revealing the hidden BID. This allows the SP to authenticate the user without inferring who the user is. While the objective is similar, our context is very different in this paper.

E. Vehicular Forensics Using Blockchain

In [29], Oham *et al.* introduces B-FICA, a Blockchain-based framework for the auto-insurance claims and adjudication process. This work takes advantage of the widespread adoption of Connected and Automated Vehicles (CAVs) to congregate data retrieved from car sensors, and communications to make undeniable claims and report them to third-parties, such as law enforcement, auto-insurance, technicians etc. B-FICA is proven to be secure mainly through tamper-proof evidence and immutable transactions using the Blockchain. The framework preserves the users privacy by using a permissioned Blockchain to limit the involved entities to transactions they are authorized to make/view.

III. BACKGROUND AND PRELIMINARIES

Before we explain the framework we provide some background on EV charging, Blockchain applications, zero-knowledge proofs, zkSNARKs, and the Pederson Commitment scheme.

A. EV Charging Procedure

Unlike traditional gas station service, EV charging can take more time (from 30 minutes to 2 hours) and thus in-advance scheduling is needed. We assume that the EVs can communicate with the EVSP and EV charging stations using some of the existing communication solutions such as LTE or IEEE 802.11p which can connect with Roadside Units (RSU) to access the Internet infrastructure. Through these communications, we follow the following protocol [30]: the EVs send charging scheduling requests that will include the EV's location, area of interest for charging, EV's state of charge (SoC), the available time slots and other related constraints. The EVSP will first need to verify that this is a genuine request and then respond with a reserved time slot and location for the EV charging station. Once the EV arrives at the charging station, it can verify that the EV is there. Note that in the current protocols the EVSP needs to collect EV information. Also, the EV user needs to make a payment via a certain method such as a credit card.

B. DApps, Blockchain, and Smart Contracts

Today most of our data is hosted in a centralized client-server architecture where the application user is the client and the server holds all the data. The problem with this architecture is that there exists a single point of failure for the servers. DApps aim to mitigate this risk by distributing data storage across all nodes.

Blockchain and *smart contracts* are building blocks of DApps. Blockchain is a decentralized ledger where each user, or node, has a ledger or list of all past blocks, where each block contains transaction(s), a timestamp, and a hash of the previous block on the chain of blocks. Since each block contains a hash of the previous block, changing the contents of one block would require changing the hash of each of the preceding blocks. Inherently blocks on the Blockchains are immutable, cannot be changed, deleted, or inserted without being verified.

Ethereum is a Blockchain platform that uses *smart contracts* to dictate the correctness of transactions. A smart contract is a block of code that allows a transaction to occur if and only if the parameters are correct. Using smart contracts on the Ethereum Blockchain allows us to make DApps, where the code is executed correctly in a decentralized manner.

C. Zero-Knowledge Proofs

Zero-knowledge proofs [11] allow a person to verify the correctness of a statement from another person without revealing the data used in the statement to itself or anybody else. In this way, the privacy of the person submitting the information is protected. zkSNARKs [10] is an efficient variant of zero-knowledge proofs that allows the prover to prove his/her knowledge in a single message. In the next subsection, we provide details of zkSNARKs that we used in our work. First, we describe the different roles involved in a zero-knowledge proof:

- **Trusted Third Party (TTP):** The TTP is in charge of registration of the user or *Prover*. The TTP creates a secret function which the Prover will solve to authenticate itself to the TTP. The TTP will also generate a smart contract with a verification key embedded in it. The contract is used by the *Verifier* to verify the generated proofs.
- **Prover:** The Prover is the client registered with the TTP. The Prover will be in charge of solving the secret function to create a witness. Using the witness, it will generate a zero-knowledge proof attesting to its knowledge of the secret function, thus, authenticating itself.
- **Verifier:** The verifier is the party that could verify the Prover's proof without learning the details of the proof or the Prover. The Verifier uses the contract from the TTP to check the correctness of a proof.

In the zero-knowledge commitment scheme, there exists a setup and verification phase. Let U denote the user and prover, ttp a trusted third party, V the verifier, and π an honestly generated proof. In order for a zero-knowledge proof to exist it must contain three properties [31]: 1) *Completeness*: V always accepts an honestly generated proof π from U ; 2) *Knowledge*: A Prover U who does not know a secret will unlikely be able to generate a valid π therefore unable to convince V ; and 3) *Zero Knowledge*: A valid π does not reveal any information about the secret. Protocol 1 provides the details for a *zero-knowledge proof of knowledge*.

Protocol 1 Zero-knowledge Commitment Scheme

1) Setup

- a) ttp generates some secret function using some knowledge U possess
- b) U inputs values satisfying the condition of the function, returning the witness to U

- c) U uses the witness to generate π_U

2) Verification

- a) U sends π_U to V , who verifies the validity of π_U
- b) V acts on behalf of ttp to grant or deny access to U , without learning the contents of π_U

D. zkSNARKs

Zero-Knowledge Succinct Non-interactive ARguments of Knowledge (zkSNARKs) are a form of zero-knowledge proofs which are distinguished by the following properties:

- 1) *Non-interactive*: a prover can prove awareness of a piece of knowledge to a verifier with a single message.
- 2) *Succinct*: verification time of the proofs is small
- 3) *Constant size*: proofs generated by zkSNARKs are the same size regardless of the complexity of the program being proven.
- 4) *Private fields/inputs*: ability to prove using private inputs that are not revealed to the verifier.

The properties mentioned above make using zkSNARKs a perfect candidate for a verification scheme using Blockchain. That is, since they are non-interactive, then the prover needs to perform only one blockchain transaction to be verified. Verification time is both short and constant, regardless of the complexity of the underlying program. Additionally, since the verifier cannot infer any details from a proof, this guarantees privacy, even in the public blockchain ledger. Finally, since the proof size is small it is feasible for proofs to be sent over most networks, in our case it is an EV transmitting the proof over a wireless link such as LTE or DSRC.

E. SE-SNARKs

For our use case it is important to have a proving scheme which is non-malleable. A malleable proving scheme means that using an existing proof, somebody can generate another valid proof for the same program. Since we use one verifying contract for all EVs, one must not be able to derive a valid proof from an existing proof. Simulation-Extractable Succinct Non-interactive ARgument of Knowledge (SE-SNARKs) [32] come with the same security benefits as zkSNARKs. Additionally, SE-SNARKs provide two imperative enhancements:

- 1) *Smaller proof sizes*: around 127 bytes, compared to [33], which have proofs that are about 288 bytes.
- 2) *Non-Malleable*: a new proof cannot be generated using a previously submitted proof.

SE-SNARKs, [32] utilize Signatures of Knowledge (SoK) [34], [35] to make their proving scheme simulation-extractable. Basically, this works by allowing the prover to create a signature based on the witness, where the signature does not reveal details of the witness, and somebody cannot look at previous signatures to make a new signature. In short, this means a prover must know a valid witness to a program in order to sign it, thus this proving scheme provides non-malleability.

F. Pederson Commitment

Pederson commitment [12] is a non-interactive verifiable secret sharing scheme. The Pederson commitment scheme allows the committer to commit to a message and send it to some verifier

without disclosing any of the commitment details to the verifier or any middleman. The committer can at a later time reveal the secret to the verifier, and the verifier can reconstruct a new commitment using the secret. If the reconstructed commitment equals the initial commitment, then the verifier is sure of the committed value. Pederson commitment is based on the difficulty of solving discrete logarithms and can be shown using the following three steps:

Setup: Let p and q denote large primes such that q divides $p - 1$, G_q is the unique subgroup of Z_p of order q , and g is a generator of G_q . Let $h \in G_q$ be chosen such that nobody knows $\log_g h$. All p, q, g, h are chosen by a verifier or trusted third party and made public.

Commit: Let m denote a secret message, or commitment, and $r \in Z_q$ be a random value, and compute:

$$c_1 = C(m, r) = g^m h^r \mod q \quad (1)$$

We will denote the result of this computation as commitment, c_1 . The committer sends c_1 to the verifier at time τ_1 .

Open: To verify the commitment the verifier receives m and r from the committer, at some time τ_2 , where $\tau_2 > \tau_1$ and computes a second commitment c_2 using the same method:

$$c_2 = C(m, r) = g^m h^r \mod q \quad (2)$$

It then proceeds to verify the authenticity of the commitment:

$$c_1 == c_2 \quad (3)$$

The work in [12] shows that the commitment c_1 reveals no information about the secret message m and nobody can open a commitment made with m using $m' \neq m$ unless they know $\log_g h$. This is called the hiding and binding property respectively.

IV. NETWORK AND THREAT MODEL

In this section we describe our Network and Threat models.

A. Network Model

Our network model assumes the communication infrastructure needed in Fig. 1. Basically, we assume that an EV can communicate with an EVSP using either LTE/4 G communications or during parking via home WiFi. This communication is assumed to be secure (i.e., providing confidentiality and integrity). EVs communication to Blockchain can follow a similar pattern. As the process can take place when the EV is moving, we assume the communication can be done via LTE/4 G or through roadside units via Dedicated Short Range Communications (DSRC) standard [36]. EVSP to Blockchain link can be wired via Internet.

B. Threat Model

The adversary in our model can see all Blockchain transactions, and can also be a nefarious EVSP. Threats pose risks to the EV that include but are not limited to location privacy leakage, physical security, loss of PII, service availability, etc. We consider the following threats in the decentralized setup as shown in Fig. 2.

- **EVSP Attack:** We assume the EVSP is an honest but curious adversary as defined in [37]. This means that the EVSP is

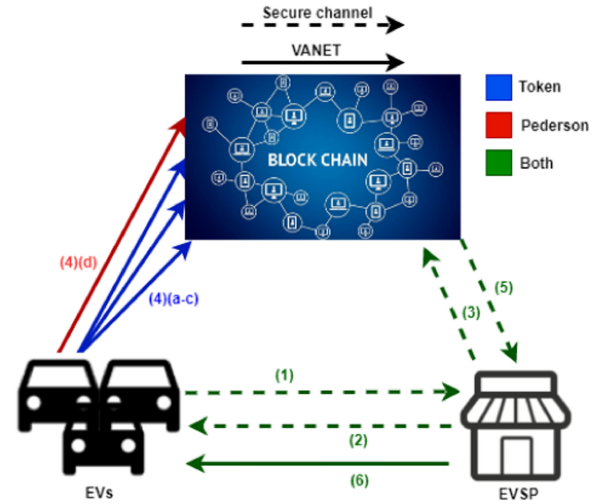


Fig. 1. Network Model: the steps indicated represent the communication between Electric Vehicles (EVs), Electric Vehicle Service Provider (EVSP), and Blockchain when scheduling charge slots.

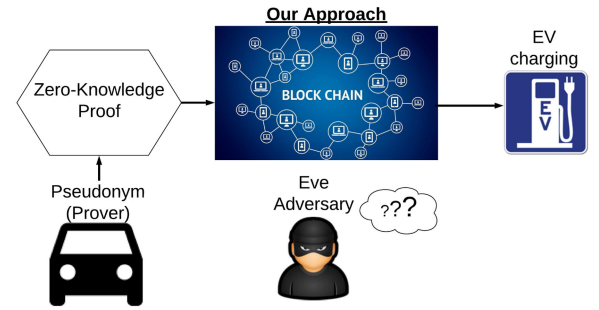


Fig. 2. Attack model for our application: **Eve** the adversary can see all Blockchain transactions, she can also be part of the EVSPs organization.

a legitimate participant in a communication protocol who will not deviate from the defined protocol but will attempt to learn all possible information from legitimately received messages. Such an EVSP can use the EV's personal information or charging habits for undesirable purposes, or distribute the data either unwillingly or with intent.

- **Public Ledger Attack:** Since the ledger is public, anyone can have access to it and analyze the transaction details of an EV with EVSP or charging stations. Thus, if an EV authenticates on the public ledger his/her credentials will be exposed.
- **Man in the Middle Attack:** A malicious adversary can capture EV's scheduling requests and analyze the packets to obtain private information.
- **Replay Attack:** A malicious adversary can use the submitted proofs to Blockchain and re-submit them to authenticate itself.
- **Charging Station Attack:** The charging station can analyze EV's request and proofs to find out private information.
- **Denial of Service Attack:** A EV can produce excessive amounts of charging periods, causing a shortage of available charging periods for other EVs.

V. PROPOSED TOKEN-BASED APPROACH

In this section, we describe our proposed framework in detail. We first start with an overview and then move onto its components.

A. Overview

To protect the EVs' privacy, we propose a novel framework that combines the zkSNARKs and the Ethereum distributed ledger. The idea is to integrate the zero-knowledge proof process in Protocol 1 to the Blockchain environment. Specifically, the elements of Protocol 1 are mapped to EV charging setting as follows: The *Prover* is the *EV*, the *TTP* is the *EVSP*, and the *Verifier* is the *Blockchain* which holds smart contracts. In our case, we assume Ethereum-based smart contracts for our approach.

Briefly, the process works as follows: The EVSP creates a secret function and passes it along with a proving key to the EVs. An EV solves the secret function to create a witness, then uses the proving key and witness to generate a proof to show that it knows the secret function EVSP created. The EV then contacts a Blockchain authentication contract and presents the proof. Once the contract verifies the proof, it generates a Service-token for the EV to be used for scheduling charging. This service-token is spent in scheduling and the EV uses a returned charging token at any charging station. In this way, the EV does not need to authenticate itself to the EVSP.

B. Registration of EVs

We assume that the EVs will register with the EVSP in advance. Basically, the EV identifies itself to the EVSP where it is looking to use the services of the EVSP. This registration is much like the registration of any web services where the EV would provide information such as name, address, EV model, etc. Registration information is not used during scheduling or charging as the EVs will remain anonymous to the EVSP. Registration assures the user is a legitimate EV, and once registered the EV will obtain the necessary information for charge scheduling, outlined in the next section.

C. Setup Phase

Once the EVs are registered, the rest of the process starts with the **Setup phase** where the EVSP generates two elements to be given to the registered EV users and one element for writing to the Blockchain as summarized in Fig. 3.

First, the EVSP generates a *secret function*, $f(x)$, to be shared with the registered users so that they can generate a *witness* and a *proof* to authenticate themselves and schedule EV charging later. $f(x)$ might employ a series of conditions for its input variables including arithmetic operators and flow control statements. Algorithm 1 provides an example of such a secret function where X is a list of 3 inputs. In this function, the 3 inputs are accumulated into a variable sum , which is eventually compared to another variable tot . If sum is greater than or equal to tot , the function returns sum , else it terminates. Note that finding the variables that satisfy the conditions of the secret function is called *witness* which happens in the **Attestation generation phase** as will be detailed next.

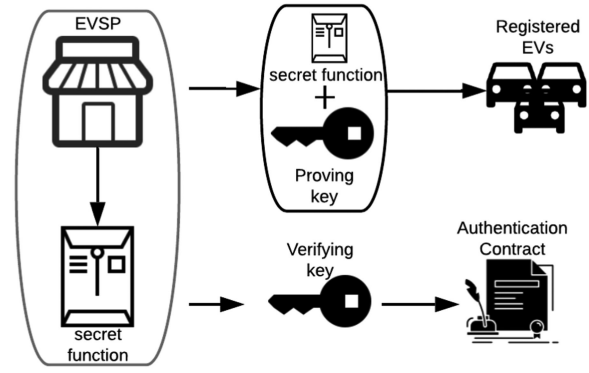


Fig. 3. Setup Phase: EVSP distributes proving keys, and the secret function to authorized users. EVSP also creates an Authentication smart contract with the verifying key embedded.

Algorithm 1: Secret Function 1.

Require: $X.length() = 3$

- 1: **procedure** MAIN(X)
- 2: $sum = 0$
- 3: $tot = 10$
- 4: **for** each integer i in X **do**
- 5: $sum = sum + i$
- 6: **if** $sum \geq tot$ **then return** sum
- 7: **else**
- 8: **terminate**

In the **Setup phase**, the EVSP also creates *Proving* and *Verifying* keys from a Common Reference String (CRS), which will be used to create and verify proofs. While the *Proving* key is sent to EV users, the *Verifying* key is sent to a Blockchain smart contract which is created for EV Authentication. This smart contract, which is shown as *Authentication Contract* in Fig. 3 is public on the Ethereum network and is used for authenticating the EV.

During the Setup phase, the EVSP deploys two tokens, *service-token* and *charge-token* from a pool of tokens maintained by it. These tokens will be used in the next phases.

D. Attestation Generation

Once an EV user is registered and receives the authentication elements, it starts the **Attestation generation phase** where the EV generates a proof that attests its knowledge of the secret function, $f(x)$.

The EV begins this process by assigning a set of variables that will satisfy the parameters of the $f(x)$. It is assumed that the EV knows $f(x)$ and thus can provide the satisfying values. This assignment of variables is called generating a *witness*. If we consider the $f(x)$ portrayed in Algorithm 1, a witness for this $f(x)$ will be any 3 variables whose sum is greater than or equal to 10 (e.g., 2, 3, 5). The witness also includes the return value (i.e., the output) based on the selected parameters, in this case, 10. Although Algorithm 1 is a simple function, we provide it as an example for making it easier for the reader to understand the concept.

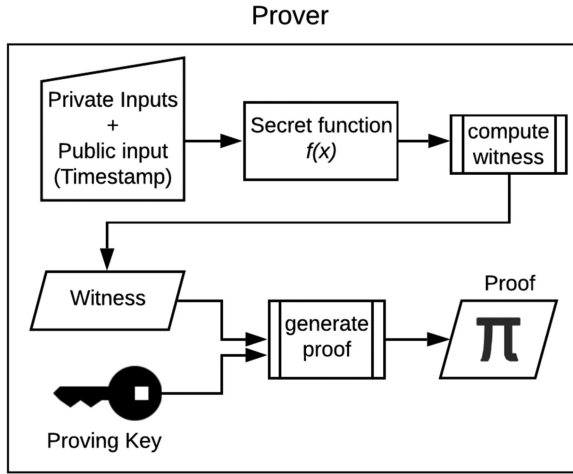


Fig. 4. Proof Generation: EV computes a witness using values that satisfy the secret function and a public inputs. With the witness and proving key, the EV generates a proof.

Next, using the witness along with the proving key, the EV generates a *proof*, namely π , attesting to its knowledge of the $f(x)$. We also append a timestamp as public input to the proof to prevent potential replay attacks. The reason for making the timestamp a public input is to allow it to act as a plaintext input to the smart contract.

This process is shown in Fig. 4. Note that all of the steps except the deployment of the *Authentication contract* up to now are performed off-chain, meaning they are not written to the Blockchain. The *Authentication smart contract* is created on the Blockchain for achieving our goal of distributed authentication.

E. Verification for Authentication

Upon possessing the valid proof, π , for $f(x)$, the EV submits his/her π to the *Authentication smart contract* on Ethereum for authenticating itself. The EV uses his pseudonym address, to interact with the contract. Importantly, the EV will generate a new pseudonym address each time it wishes to schedule a charging service with the EVSP. Generating new pseudonym addresses is easy (performed almost instantly), free of cost, and done off-chain. Along with π , an array of public inputs including the output of $f(x)$ (given the witness values), and optional public parameters to $f(x)$ are also passed to the Blockchain. Once the authentication is successful, the smart contract issues a *service-token* that belongs to the EV. Note that this token is assigned to the same pseudonymous address that was used to interact with the smart contract by the EV. The authentication process is shown in Fig. 5.

F. Charging Scheduling

The *service-token* received after authentication is later used by the EV for scheduling the charging on another smart contract which is referred to as *scheduling smart contract*. Basically, the EV submits the *service-token* as well as its desired available time slots to the scheduling smart contract on Ethereum. The address

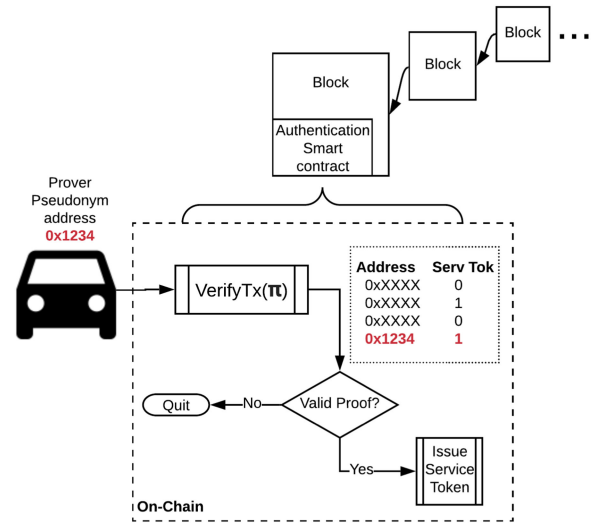


Fig. 5. Authentication: EV submits a proof to the authentication smart contract. If the input is valid, then the smart contract returns a service-token to the EV's pseudonym address.

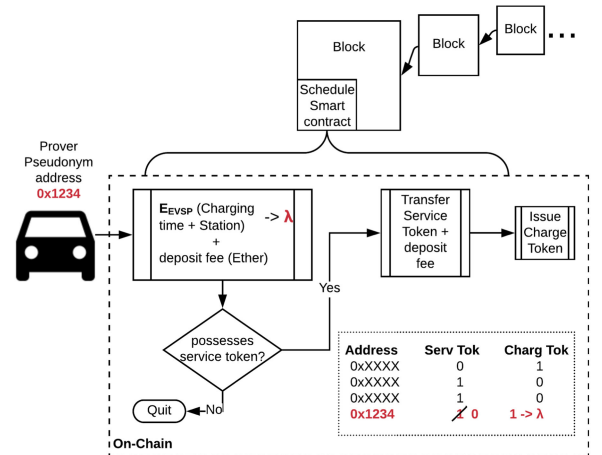


Fig. 6. Scheduling: EV spends its service-token on a scheduling contract to receive a charging-token for EV charging.

used to submit this *service-token* is by default the address which received it from the authentication smart contract.

The scheduling information is sent encrypted with the *public key* of the EVSP to hide the charging details from the public which is depicted as λ in Fig. 6. EVSP also interacts with the scheduling smart contract and hence can receive the encrypted scheduling information to properly schedule a time slot for the spender of that service-token. The scheduling smart contract returns a new *charging-token* to the EV, which can be spent during its scheduled charging slot for charging. During this phase, the EV also sends a deposit fee in the form of Ethereum which serves the purpose of a security deposit for charging. The deposit fee is the same for everyone, this way there can be no pattern tracing to identify a specific EV. Without this fee, scheduling would be almost free and this could be abused

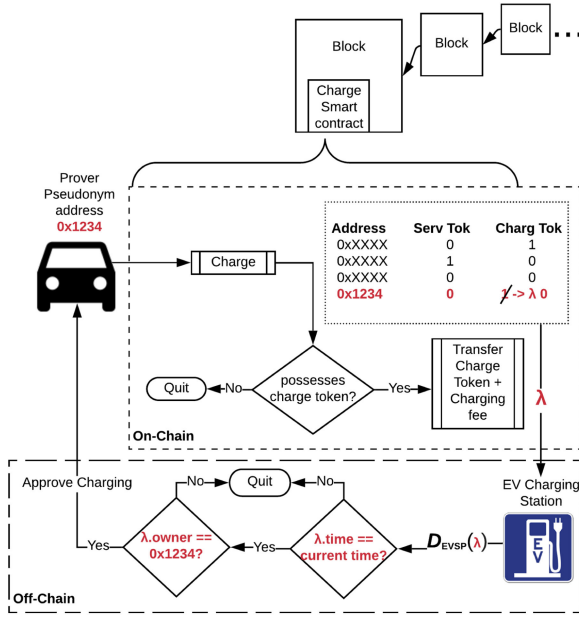


Fig. 7. Charging: EV uses his/her charging-token at a charging station during its allotted period.

by malicious EVs to schedule a large number of fake charging appointments. This process is shown in Fig. 6.

G. EV Charging Station Verification

When the EV arrives at the charging station at its allocated charging time slot, the charging station will receive the charging-token at its address. Then the EV charging station verifies with the EVSP that the EV which submitted the charging-token is the currently scheduled EV. Basically, it contacts the smart contract with the token and receives the information about the ID and time (encrypted) to make a comparison.

Note that with the charging-token, the EV also makes a payment (in addition to deposit fee) in the form of Ether cryptocurrency to the charging station. Using the proposed Blockchain framework allows the EV to pay anonymously in this step which is important for privacy. Following the verification of the charging-token and sufficiency of funds, the EV will proceed to charge and send a message to EV. We assume there will be a communication channel (i.e., cable or wireless) that will run the protocol between the EV and the charging station to send this approval message. The charging fee paid by the EV in this phase represents payment for the amount of charging it needs. Fig. 7 demonstrates this charging verification process.

VI. PEDERSON COMMITMENT APPROACH

The aforementioned approach provides zero-knowledge and anonymity when scheduling charging for EVs. However, there is still some overhead associated with the use of tokens in performing authentication, scheduling and charging on Ethereum in terms of efficiency and transaction costs.

Therefore, as an alternative to tokens, we propose adapting the Pederson commitment scheme which is a method of sharing a secret that is both binding and hiding. In our privacy framework,

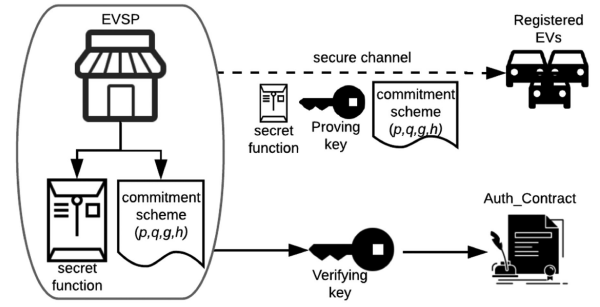


Fig. 8. Improved Setup: EVSP distributes proving keys, the secret function and the Pederson Commitment parameters p, q, g , and h to authorized users. EVSP also creates an *Auth_Contract* with the verifying key embedded.

Algorithm 2: Secret Function 2.

```

1: Procedure main (private  $X$ ) returns (bool result)
2:    $hash\_image_1 = H(X)$ 
3:    $hash\_image_2 = b94d27b9934...$ 
4:   if  $hash\_image_1 == hash\_image_2$  then return 1
5:   else return 0
6:   terminate

```

we propose using this commitment scheme to allow an EV to commit to a scheduled charging slot. This improves the efficiency of the framework and reduces the cost by reducing the transactions performed on-chain, including the removal of the scheduling, and charging transaction completely. In the following sections, we describe the changes needed at each step in the proposed framework.

A. Setup Phase

The Pederson Commitment parameters along with $f(x)$, and proving key are given to registered EVs, as shown in Fig. 8.

For this approach, the EVSP must generate the required parameters for a Pederson Commitment scheme. That is, the EVSP will perform the setup phase of the Pederson Commitment scheme which is described in Section III-F. The outcome of this setup step will be the public Pederson Commitment parameters, namely p, q, g, h . These parameters are sent to registered EVs to be used in generating a commitment that contains the requested schedule for charging.

In this approach, we implement a different $f(x)$ which can be proven to be difficult enough that it is cryptographically secure for our use case. In this case, we use a $f(x)$ that requires the prover to prove his/her knowledge of the preimage of a hash function. Thus, the difficulty of providing an invalid input to solve the secret function lies in the difficulty of generating an X' such that: $X' \neq X \& H(X') == H(X)$.

The new $f(x)$ is shown in Algorithm 2. It accepts a private input X and returns a boolean value, true or false. It is true if the hash of a user's input, $hash_image_1$, is equal to the hash of a secret value, $hash_image_2$, known only to the EVSP and authorized EVs.

During the setup phase, the EVSP also deploys a single smart contract, called *Auth_contract* for brevity. The *Auth_contract* performs the following functions in order:

- 1) Performs authentication of zkSNARK π granting access to authorized EVs.
- 2) Accepts a Pederson Commitment containing a scheduled charging slot chosen by the EV.
- 3) Accepts a message encrypted with the public key of the EVSP that states the desired charging slot.
- 4) Accepts payment in the form of Ethereum for the associated charging appointment.

The *Auth_contract* posts an event whenever an EV authenticates and schedules charging, from which the EVSP uses the private key to decrypt the message and book the charging slot. The charging appointment will be available for the EV who can provide the secrets to open the Pederson Commitment. The setup phase ends when the EVSP sends the Pederson Commitment parameters along with the $f(x)$ and proving key to registered EVs, as shown in Fig. 8.

B. Attestation Generation

Next, the EV must attest his/her knowledge to $f(x)$ and the parameters of the Pederson Commitment. In this case, attesting knowledge to $f(x)$ means he/she knows the preimage of a hash image using $H(input)$. The EV inputs the preimage to $f(x)$ and if:

$$H(input) == hash_image$$

then the EV will receive a witness. It is important to note that the input is private and not made public during the authentication *on-chain*. In this approach, we again utilize the notion of a public input that is a timestamp to avoid the reuse of the same proof. Combining the generated witness and the proving key received from the EVSP during setup, the EV will generate a proof, π . The proof generation process is shown in Fig. 4.

Additionally, the EV must attest his/her knowledge of the Pederson Commitment parameters and thus be able to use these parameters for creating his/her commitment. A security benefit of the Pederson Commitment is that even if the parameters, namely p, q, g, h , were to be released to the public or published on a public site, an adversary cannot use this information for an attack on the commitment. For our sake, the EVSP sends the Pederson Commitment parameters along with the proving key and $f(x)$ as the combined size of the parameters is only 188 bytes and thus does not introduce heavy overhead.

The EV uses equation 1 to create a commitment. In this case, m , the secret, is a *unsigned integer* representing the charging slot chosen. Additional digits are appended to represent the charging station where the EV wishes to schedule the charging. The other input to a commitment is a random value, r , which is randomly generated by the EV and stored for opening the commitment in a later period. Using m, r, p, q, g , and h the EV calculates a commitment, c_1 , of order q using Equation 1 detailed in Section III.

C. Authentication and Commitment

At this point, the EV has a π , and a commitment, c_1 . Additionally, the EV knows what charging slot it would like to schedule, as this is contained within c_1 . Knowing the charging slot, the EV writes a message, λ , and encrypts it using the public key of the EVSP. This message is designed to allow the EVSP to book the

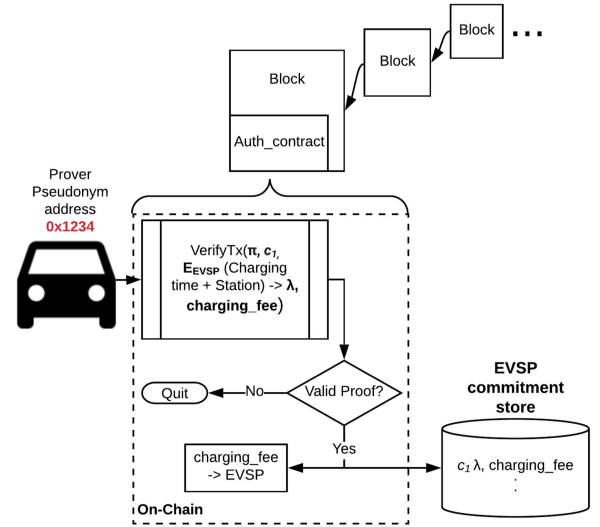


Fig. 9. Authentication and Commitment: EV submits a proof π , a commitment c_1 , a message λ containing charging schedule encrypted using the public key of the EVSP, and any charging fees in the form of Ethereum.

charging station for that charging period. Now possessing these three parameters, the EV uses a pseudonym address to send them to the *auth_contract*. The EV also includes any payment in the form of Ethereum that is required for the charging, as well as public inputs and the output to $f(x)$.

Assuming the EV has provided a valid π , he/she would have committed to that charging slot and no longer needs to interact with the Blockchain. The process of authenticating and committing to an EV charging schedule is shown in Fig. 9.

D. Scheduling

The *auth_contract* verifies the π triggering an event, passing λ and c_1 to the EVSP. The EVSP decrypts λ to book the charging station for that period and associates it with the c_1 and the payment. For maintaining the commitments and associated charging details, we store them *off-chain* in a database called *EVSP commitment store*. The charging station will retrieve the commitments from this storage for use in authenticating the EV for charging.

E. Charging Station Verification

The final step to this approach is verifying the EV when it arrives for charging during its assigned period. When the EV arrives at the charging station, it uses a secure channel to transmit the secrets to open the *Pederson Commitment*, which is the m and r that will open c_1 . Using m and r , the EVSP creates a new commitment c_2 using Equation 2 and compares it to c_1 that is associated with that charging time using Equation 3. If the commitments are equal then the EVSP is sure the EV that has arrived is the same one that committed to that charging slot when it authenticated using the *auth_contract*. During this step, the EVSP also verifies that the current c_1 represents the scheduled EV. This process is shown in Fig. 10.

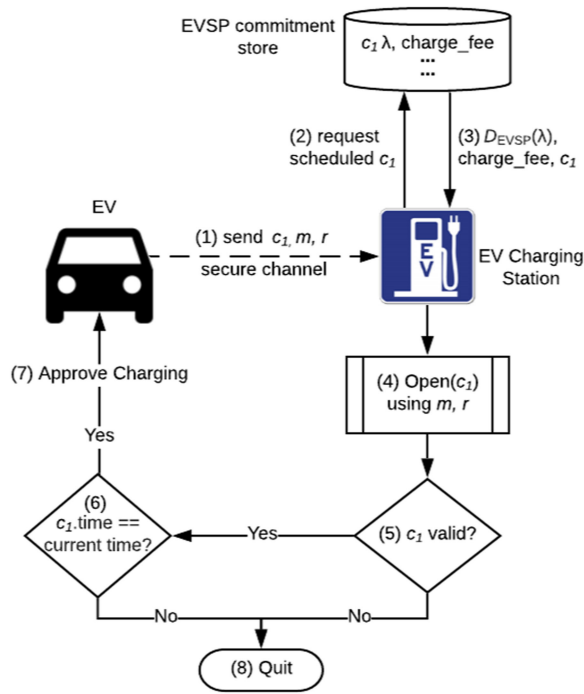


Fig. 10. Charging: EV sends its original commitment c_1 , and the secrets to open it, m and r using a secure channel. Then, the charging station retrieves the scheduled commitment and compares it to the commitment received from the EV. If the commitments are equal, it proves the EV that has arrived is the scheduled EV. Next, the EVSP opens c_1 using the secrets provided by the EV. If the c_1 is valid and contains the correct time then charging is approved.

VII. SECURITY AND PRIVACY ANALYSIS

A. Security Analysis

In this subsection, we provide an analysis of how our approach addresses the threats mentioned in our Threat model. Within each analysis, we mention how it relates to a token-based approach or Pederson commitment if needed.

EVSP Attack: Our framework mitigates EVSP attack risk by authenticating EVs to the EVSP through merely a zero-knowledge proof, thereby, hiding the identity of the EV. In our framework, the EVSP is only aware that an authorized EV is requesting to schedule a charging slot. EVSP cannot gain any personal information from the EVs authentication process including charging habits, location, and personal information.

Public Ledger Attack: Even if different pseudonyms are used for each transaction, tracking the flow of funds may expose some privacy. A fund flow contains a variety of information such as public addresses and transferred amounts (e.g., transferred amount pattern). Then, a flow-based analysis enables eavesdroppers to identify and monitor interacting parties. Using a similar analysis, an eavesdropper, for our token-based approach, can try to match different token transfers to a single EV identity even though the EV uses different pseudonyms. We claim that this is not possible in our approach, because the token transfer to various pseudonym addresses is essentially the same (including the deposit fee) for all cases since the amount for the transaction will be the same (i.e., we use the same token). Thus, an adversary cannot differentiate among these and link certain pseudonyms. Here, another possible passive attack by the adversary could

be doing timing analysis on the public ledger to try to match different pseudonyms to an EV. This is also not possible due to two reasons: 1) the token transfers are sporadic (i.e., in order of days); and 2) since the EV switches to a new pseudonym for every new charging service regularly, a consistent pattern is not possible to identify. We also note that a passive adversary can not obtain details of scheduling requests either, since the request is encrypted with the public key of the EVSP and therefore malicious actors can not obtain details of the scheduling event. In the case of Pederson, opening and/or finding a c_1 which is equal to the commitment submitted is sufficiently difficult that an adversary cannot find the secrets to recreate the commitment.

Man in the Middle Attack: Since each EV sends the scheduling information encrypted with the EVSP's public key to the Blockchain, even if an adversary could capture this packet, he/she will not be able to access its contents. Also, the ID of the EV is anonymous and changing regularly. Assuming, the EVSP is compromised by the adversary and the schedule is revealed, then the adversary will only learn of a time and location of EV charging. In this case, the adversary will learn nothing about the identity of the EV and linking this EV to future charging will be impossible.

Replay Attack: We prevent proofs from being replayed by adding a timestamp as input during the attestation generation phase. The timestamp is a public input and therefore it is submitted to the smart contract when verifying the proof. The authentication contract then verifies the current time. If the current time is within the proof validity period, then the proof is valid. Otherwise, it is denied.

Charging Station Attack: For the token-based approach, when the EV arrives for charging, it submits only a charging-token with a pseudorandom Ethereum address to get charged. During this transaction, no details that can lead to the ID of the EV are revealed. In the case of Pederson, when the EV arrives, it submits the c_1 , m , and r . The EV does not identify itself to the EVSP since it is sufficient that it knows the scheduled c_1 and the secrets to open the c_1 .

Denial of Service Attack: In the token-based approach, EVs can schedule as many charging as they want. However, this requires a deposit fee payment in advance, which will be written on the charging token and thus an adversary will have to limit its scheduling requests. Therefore, DoS attempts to make bogus scheduling will not be possible. For Pederson, payment for scheduling charging is sent upfront during authentication and commitment. Thus, excess scheduling would be a costly attack that will not be attractive.

B. Privacy Analysis

The information regarding an EV's ID, choices, location, etc. was the main concern for exposure. Through our security analysis, we have shown that this information will not be exposed to parties that are not trusted. Briefly: 1) EVSP will not have access to any of this information while it can still authenticate EVs via zkSNARKs; 2) Charging station will only know the ID but this is anonymous, so no other information will be revealed; and 3) Any adversary capturing traffic or accessing the public ledger will not be able to obtain any scheduling or location information and do any matching of IDs. Therefore, the proposed

approaches enable the privacy of the EVs in a decentralized manner.

VIII. EVALUATION

A. Experimental Setup

Our framework for evaluation is built primarily over Zokrates [13] which is an all-in-one tool used to implement zkSNARKs into a distributed ledger. Zokrates was used for generating the witness and proofs, using two commands, compute-witness and generate-proof respectively. We evaluate the performance of these two phases on an EV by using a Raspberry PI 3 model B V1.2 (Quad Cortex A53 @ 1.2 GHz, 1 GB SDRAM, 4000 Hz Videocore IV). We evaluate the performance of the protocol based on time and the size of the proof.

Additionally, we used Zokrates for implementation onto the Ropsten Test network which was used for on-chain transactions through smart contracts. Ropsten is an Ethereum network simulator and is meant to mirror the Ethereum main network. To create a smart contract on Ethereum, we used the export-verifier command of Zokrates which generates our smart contract written in Solidity, which is the native language for writing smart contracts on Ethereum. The verifying key is embedded into the Ethereum smart contract used for authenticating the EVs. For the Pederson commitment approach, we use a similar smart contract that verifies the π , stores c_1 , accepts payment, and triggers an alert to the EVSP. It does not rely on token contracts.

For the Pederson commitment, we reduced the computation time by using the multiplicative property of the modulo function. Basically, Equation 1 and 2 are expanded to the following equation:

$$c = C(m, r) = (g^m \bmod q)(h^r \bmod q) \bmod q \quad (4)$$

Using Equation 4, we were able to compute a commitment that is 256 bits in under 10 ms and 1048 bits in less than 45 ms. Since verifying the commitment is done so quickly if a charging station already has the scheduled commitment, we omitted it from the total overhead of the EV in the experiments.

B. Performance Metrics and Benchmarking

The following metrics are considered when evaluating the overhead of our approach:

- **Gas:** Ethereum runs on the notion of *gas*, which is the fee paid by the user when interacting with a smart contract. The amount of gas required by a smart contract transaction is determined by the complexity of the transaction. However, there is a gas limit per transaction block on the Ethereum. This gas limit determines how many transactions will fit on each block.
- **Cost:** This is the cost of gas in terms of Ethereum cryptocurrency which is *ether*. The user has the option to choose his/her transaction verification speed by specifying how much ether he/she will pay per gas unit to the verifier. We used three verification speeds: *slow*: 1×10^{-9} Ether/gas or 1 Gwei; *medium*: 3×10^{-9} Ether/gas or 3 Gwei; *fast*: 2×10^{-8} Ether/gas or 20 Gwei. We also converted this ether cost to dollars using its approximate exchange rate at the time of writing (\$144/Ethereum). This exchange rate can vary depending on the value of Ethereum.

TABLE I

WITNESS AND PROOF GENERATION TIME IN SECONDS WITH FUNCTIONS THAT HAVE 4, 8, AND 16 INPUTS, NUMBER OF CONSTRAINTS, AND THE SIZE OF PROOF IN BITS

# of Inputs	# of Constraints	Compute-witness (seconds)	Proof generation (seconds)	Proof size (bits)
4	4	0.007	0.069	1019
8	4	0.008	0.071	1019
16	4	0.015	0.082	1019
16	16	0.010	0.079	1019
16 - sudoku	695	0.081	0.140	1019
4 - SHA256	56,985	0.606	14.38	1019

TABLE II

SMART CONTRACT DEPLOYMENT OVERHEAD - 2 INPUTS - TOKEN APPROACH

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	15.79	5,479,043	0.00548	0.79
medium	11.95	5,479,043	0.0164	2.36
fast	11.54	5,479,043	0.110	15.78

- **Time:** This is the time required to verify a transaction on the Blockchain. While it may vary, we show the average time over 30 trials for different verification speeds, which can be chosen by the Blockchain user (i.e., slow, medium, fastest).

For the token-based approach, implementing our framework on Ethereum has four primary transactions: the contract deployment, the verification of proofs (i.e. authentication), scheduling, and charging. We benchmarked these costs in terms of gas, time, and money.

To implement the Pederson commitment approach, we only need to deploy the *auth_contract* and measure its proof-verification costs. The cost of deployment and verification are also benchmarked in terms of gas, time, and money.

C. Evaluation Results

1) *Computation Overhead on EVs:* We first evaluated the feasibility of generating a witness and proof on an EV using $f(x)$ with different numbers of inputs. We used the SHA256 hash function for the Pederson commitment. Our experiments for the token-based approach were done using a $f(x)$ with 16 private inputs and another $f(x)$ with 2 private inputs. Each of these $f(x)$ also has 1 public input, the timestamp. For the Pederson Commitment approach, we used a $f(x)$ with 4 private inputs which represent the preimage of a SHA256 hash, and 1 public input, the timestamp. In the experiments, we repeated each of the steps 30 times and at random times of the day.

The results of the experiments are shown in Table I. It is shown that generating a proof for the SHA256 $f(x)$ can take up to 15 seconds. Since the EV can compute and generate a proof at any time, it is still feasible for an EV to generate a proof(s) for a future charge scheduling. Additionally, the size of the proof is sufficiently small (i.e., 1019 bits) that sending the proof over a V2I communication will not cause any delay for the EV.

2) *Overhead of Contract Deployment:* Next, we assessed the overhead of contract deployment on the Blockchain. Table II and III show the cost of the token-based approach contract deployment for 2 and 16 inputs $f(x)$ respectively. Table IV shows the cost of the Pederson based approach *auth_contract*. The results indicate that if slow and medium options are chosen, the costs are almost negligible. Given that this deployment is done only once by the EVSP, time is not an issue here and thus

TABLE III
SMART CONTRACT DEPLOYMENT OVERHEAD - 16 INPUTS - TOKEN APPROACH

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	22.86	5,479,183	0.00548	0.79
medium	15.64	5,479,183	0.0164	2.36
fast	5.29	5,479,183	0.110	15.78

TABLE IV
SMART CONTRACT DEPLOYMENT OVERHEAD - PEDERSON APPROACH

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	41.41	1,616,521	0.00162	0.23
medium	24.32	1,616,521	0.00485	0.70
fast	14.12	1,616,521	0.0323	4.65

TABLE V
AVERAGE AUTHENTICATION OVERHEAD - 2 INPUTS

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	24.28	1,693,250	0.00169	0.24
medium	24.08	1,693,250	0.00508	0.73
fast	19.11	1,693,250	0.0339	4.88

TABLE VI
AVERAGE AUTHENTICATION OVERHEAD - 16 INPUTS

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	15.97	1,693,058	0.00169	0.24
medium	15.72	1,693,058	0.00508	0.73
fast	14.45	1,693,058	0.0339	4.88

slow mode would be a viable option. Note that both approaches do not require a contract for each EV and thus saves a lot of time and cost.

3) *Authentication, Scheduling, and Charging Overhead:* Next, we assessed the overhead for each EV through different contracts using the token-based approach. First, we looked at the authentication overhead to see if a different number of inputs would make a difference. Table V and VI shows the cost of verification of a $f(x)$ with 2 and 16 private inputs respectively, using the authentication smart contract. The results indicate that the gas cost associated to do the authentication transaction is the same for both cases and thus the number of inputs does not have an impact on the overhead. This mainly due to the nature zkSNARKs which does not differentiate between varying computations. The verification times, however, differ due to two reasons: 1) when there is more gas used for a transaction, the miners process it more quickly since the fee they get will be higher; 2) there is volatility in Ethereum network transactions depending on the specific time they performed. Therefore, we see that with 16 inputs, the verification times are slightly reduced. However, overall we can see that on average the authentication time does not take more than 24 seconds in any case.

After authentication, we looked at the overhead for scheduling and charging using the token-based approach. However, since these processes do not depend on any varying input, we conducted a single experiment. Tables VII and VIII show the scheduling and charging smart contract overheads per EV. We can see from these tables that the costs are very small and almost negligible. The transaction times also do not change much compared to authentication.

For our Pederson Commitment approach, the EV only needs to authenticate and commit his/her scheduled charging time to

TABLE VII
AVERAGE SCHEDULING OVERHEAD (TOKEN APPROACH)

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	18.03	59,061	0.0000591	0.015
medium	11.52	59,061	0.000354	0.0510
fast	7.24	52,439	0.00105	0.151

TABLE VIII
AVERAGE CHARGE VERIFICATION OVERHEAD (TOKEN APPROACH)

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	19.77	19,374	0.0000194	0.0049
medium	10.95	19,374	0.000116	0.0167
fast	10.51	34,374	0.000687	0.0990

TABLE IX
TOTAL SCHEDULING OVERHEAD - PEDERSON APPROACH

	Time (s)	Gas Used	Cost (Ether)	Cost (\$)
slow	37.60	875,287	0.000875	0.13
medium	35.83	875,287	0.00263	0.38
fast	20.26	875,287	0.0175	2.52

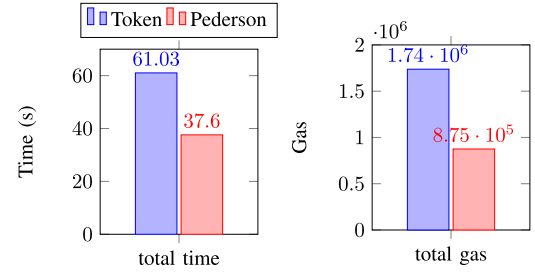


Fig. 11. Total time and gas overhead on EV using Token and Pederson approaches

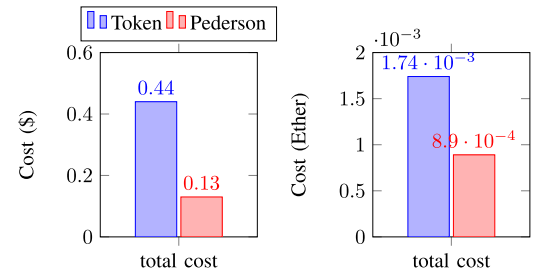


Fig. 12. Total cost in Ether and Dollar using Token and Pederson approaches. (144\$/Ether)

the *auth_contract*. Table IX shows the cost of authenticating and committing to a charge schedule. Besides proof generation and schedule commitment, this is the only overhead for the EV when scheduling the charging using the Pederson approach.

4) *Putting It Altogether:* To put it into a better perspective, in Figs. 11 and 12 we compare both of our approaches in terms of all our benchmarks.

These results indicate that overall the Pederson approach performs nearly twice as good as the token-based approach when considering the cheapest option. The total time overhead is 37.60 seconds while it is 61.03 for the token-based approach. Also, the total monetary cost is \$0.13 in the Pederson approach compared

to \$0.44 in the token-based approach. Additionally, since the EV only needs to submit one *on-chain* transaction this reduces overall interaction between EV and the Blockchain.

IX. CONCLUSION

In this paper, we introduced a framework to preserve the privacy of EVs during their charging process by combining the concept of zero-knowledge proofs and Blockchain smart contracts. We first introduced a token-based method where we used different smart contracts and tokens to conduct authentication, scheduling, and charging without relying on any other third parties. Then, we presented an improved version of our approach through the use of a Pederson Commitment rather than the token mechanism. For efficiency, in both cases, we were able to use a single smart contract that will serve all the EVs. Through security analysis, we showed that our approach will not expose any information to the involved parties such as the EVSP, public entities, and the EV Charging stations. We also implemented both approaches of this framework using Zokrates. The results indicated that the overhead of the overall process in terms of time and Ethereum cost is affordable for use in real-life applications. Nevertheless, the Pederson approach outperforms the token-based approach both in terms of time and cost overhead.

REFERENCES

- [1] M. Miller, *The Internet of Things: How Smart TVs, Smart Cars, Smart Homes, and Smart Cities are Changing the World*. London, U.K.: Pearson Education, 2015.
- [2] A. Rahim *et al.*, "Vehicular social networks: A survey," *Pervasive Mobile Comput.*, vol. 43, pp. 96–113, Jan. 2018.
- [3] E. Bulut, M. Kisacikoglu, and K. Akkaya, "Spatio-temporal non-intrusive direct V2V charge sharing coordination," *IEEE Trans. Veh. Technol.*, vol. 68, no. 10, pp. 9385–9398, Oct. 2019.
- [4] J. Isaak and M. J. Hanna, "User data privacy: Facebook, cambridge analytica, and privacy protection," *Comput.*, vol. 51, no. 8, pp. 56–59, Aug. 2018.
- [5] V. Naidu, K. Mudliar, A. Naik, and P. P. Bhavathankar, "A fully observable supply chain management system using block chain and IoT," in *Proc. 3rd Int. Conf. Convergence Technol.*, Apr. 2018, pp. 1260–1264.
- [6] J. Chen, "Devify: Decentralized internet of things software framework for a peer-to-peer and interoperable IoT device," *SIGBED Rev.*, 2018. [Online]. Available: <http://doi.acm.org/10.1145/3231535.3231539>
- [7] Z. Ma, M. Jiang, H. Gao, and Z. Wang, "Blockchain for digital rights management," *Future Gener. Comput. Syst.*, vol. 89, pp. 746–764, Dec. 2018. [Online]. Available: <http://www.sciencedirect.com/science/article/pii/S0167739X18301614>
- [8] C. Oham, R. Jurdak, S. S. Kanhere, A. Dorri, and S. Jha, "B-FICA: Blockchain based framework for auto-insurance claim and adjudication," in *Proc. IEEE Int. Conf. Internet Things (iThings) Green Comput. Commun. Cyber, Physical Social Comput. Smart Data*, Jul. 2018, pp. 1171–1180.
- [9] G. Wood *et al.*, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, 2014. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [10] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again," in *Proc. 3rd Innovations in Theoretical Computer Science Conference, (ITCS '12. Series)* New York, NY, USA: ACM, 2012. [Online]. Available: <http://doi.acm.org/10.1145/2090236.2090263>
- [11] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," in *Proc. 17th Annu. ACM Symp. Theory Comput.*, 1985, pp. 291–304.
- [12] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. Annu. Int. Cryptology Conf.*, 1991, pp. 129–140.
- [13] J. Eberhardt and S. Tai, "Zokrates-scalable privacy-preserving off-chain computations," in *Proc. IEEE Int. Conf. Blockchain.*, 2018, pp. 1084–1091.
- [14] W. Han and Y. Xiao, "Privacy preservation for V2G networks in smart grid: A survey," *Comput. Commun.*, vol. 9192, pp. 17–28, Oct. 2016.
- [15] H. Li, G. Dan, and K. Nahrstedt, "Portunes: Privacy-preserving fast authentication for dynamic electric vehicle charging," in *Proc. IEEE Int. Conf. Smart Grid Commun.*, Nov. 2014, pp. 920–925.
- [16] F. Yucel, K. Akkaya, and E. Bulut, "Efficient and privacy preserving supplier matching for electric vehicle charging," *Ad Hoc Netw.*, vol. 90, Jul. 2019, Art. no. 101730.
- [17] D. Chaum, "Blind signatures for untraceable payments," in *Advances in Cryptology*. Berlin, Germany: Springer, 1983.
- [18] T. W. Chim, S.-M. Yiu, L. C. Hui, and V. O. Li, "Privacy-preserving advance power reservation," *IEEE Commun. Mag.*, vol. 50, no. 8, pp. 18–23, Aug. 2012.
- [19] S. Jeong, N. Dao, Y. Lee, C. Lee, and S. Cho, "Blockchain based billing system for electric vehicle and charging station," in *Proc. Tenth Int. Conf. Ubiquitous Future Netw.*, Jul. 2018.
- [20] W. A. Amiri, M. Baza, K. Banawan, M. Mahmoud, W. Alasmay, and K. Akkaya, "Privacy-preserving smart parking system using blockchain and private information retrieval," *CoRR*, 2019. [Online]. Available: <http://arxiv.org/abs/1904.09703>
- [21] F. Knirsch, A. Unterweger, and D. Engel, "Privacy-preserving blockchain-based electric vehicle charging with dynamic tariff decisions," *Comput. Sci.-Res. Develop.*, pp. 71–79, 2018.
- [22] M. Pustiek, A. Kos, and U. Sedlar, "Blockchain based autonomous selection of electric vehicle charging station," in *Proc. Int. Conf. Identification, Inf. Knowl. Internet Things*, Oct. 2016, pp. 217–222.
- [23] E. B. Sasson *et al.*, "Zerocash: Decentralized anonymous payments from bitcoin," in *Proc. IEEE Symp. Secur. Privacy*, 2014, pp. 459–474.
- [24] A. Naveh and E. Tromer, "Photoproof: Cryptographic image authentication for any set of permissible transformations," in *Proc. IEEE Secur. Privacy Symp.*, 2016, pp. 255–271.
- [25] H. Wu, W. Zheng, A. Chiesa, R. A. Popa, and I. Stoica, "DIZK: A distributed zero-knowledge proof system," in *Proc. 27th USENIX Secur. Symp.*, Baltimore, MD, USA, 2018, pp. 675–692.
- [26] G. Zyskind, O. Nathan, and A. Pentland, "Decentralizing privacy: Using blockchain to protect personal data," in *Proc. IEEE Secur. Privacy Workshops*, May 2015, pp. 180–184.
- [27] M. Baza, N. Lasla, M. Mahmoud, G. Srivastava, and M. Abdallah, "B-ride: Ride sharing with privacy-preservation, trust and fair payment atop public blockchain," *CoRR*, 2019. [Online]. Available: <http://arxiv.org/abs/1906.09968>
- [28] H. Gunasinghe and E. Bertino, "Privbiomtauth: Privacy preserving biometrics-based and user centric protocol for user authentication from mobile phones," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 4, pp. 1042–1057, Apr. 2018.
- [29] C. Oham, R. Jurdak, S. S. Kanhere, A. Dorri, and S. K. Jha, "B-FICA: Blockchain based framework for auto-insurance claim and adjudication," *CoRR*, 2018. [Online]. Available: <http://arxiv.org/abs/1806.06169>
- [30] B. Roberts, K. Akkaya, E. Bulut, and M. Kisacikoglu, "An authentication framework for electric vehicle-to-electric vehicle charging applications," in *Proc. IEEE 14th Int. Conf. Mobile Ad Hoc Sensor Syst.*, 2017, pp. 565–569.
- [31] H. Mayer, "zk-SNARK explained: Basic principles," 2017.
- [32] J. Groth and M. Maller, "Snarky signatures: Minimal signatures of knowledge from simulation-extractable SNARKs," in *Advances in Cryptology – CRYPTO 2017*, J. Katz and H. Shacham, Eds. Cham: Springer, 2017, pp. 581–612.
- [33] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology – EUROCRYPT 2016*, M. Fischlin and J.-S. Coron, Eds. Berlin, Heidelberg: Springer, 2016, pp. 305–326.
- [34] J. Camenisch and M. Stadler, "Efficient group signature schemes for large groups," in *Proc. Annu. Int. Cryptology Conf.*, 1997, pp. 410–424.
- [35] M. Chase and A. Lysyanskaya, "On signatures of knowledge," in *Proc. Annu. Int. Cryptology Conf.*, 2006, pp. 78–96.
- [36] F. C. Commission, "FCC 99-305," Oct. 1999.
- [37] A. Paverd, A. Martin, and I. Brown, "Modelling and automatically analysing privacy properties for honest-but-curious adversaries," Univ. Oxford, Oxford, U.K., Casper privacy report, 2014.